Two-Colour Zig-Zag Numberlink on Rectangles: A Forbidden-Pattern Characterization, a Constructive Polynomial-Time Algorithm, and a Lean Verification

TBD

Draft, October 2026

Abstract

In Zig-Zag Numberlink, pairs of endpoints in a grid must be joined by vertex-disjoint paths that together visit every cell. The problem is NP-complete when the number of pairs is unbounded (Adcock et al.), and for a single pair it is the Hamiltonian path problem in rectangular grid graphs, solved by Itai, Papadimitriou and Szwarcfiter. We settle the case of two pairs on rectangles. If both sides of the rectangle are at least 11, an instance is solvable if and only if none of a finite catalogue of forbidden patterns occurs; the catalogue consists of a parity condition, an alternation condition on the border, a handful of local corner and edge patterns, 65 listed corner configurations, and a rule that propagates forced routes out of the corners before testing alternation. If a side is at most 10, a frontier dynamic program decides the instance in time linear in the long side. The characterization is constructive: solutions are built by deleting empty lines, splitting the rectangle by straight cuts, and splicing the pieces back together, in time $O((R+C)\,RC)$. Both directions of the characterization and the correctness of the dynamic program are verified in Lean 4. The finite case analyses (window certificates for the corner configurations, and a check of all irreducible instances with sides 11–22) are carried out by the Lean compiler via `native_decide`. We describe the algorithm, the catalogue, the proof structure, the compromises made in the formalization, and implementations in Python, JavaScript and C that produce identical output.

1 Introduction

Numberlink is a pencil puzzle: given pairs of numbered cells in a grid, connect each pair by a path along edge-adjacent cells so that no two paths share a cell. The *zig-zag* variant (also popular as the game *Flow*) additionally requires every cell of the grid to be used. Adcock et al. [1] showed that Zig-Zag Numberlink is NP-complete. With a single pair, it asks for a Hamiltonian path between two given cells, which Itai, Papadimitriou and Szwarcfiter (IPS) characterized for rectangles [3]: a simple colour condition plus three small families of exceptions.

We study the next case: two pairs, on an $R \times C$ rectangle. The instance has endpoints s_0, t_0 of colour A and s_1, t_1 of colour B; a solution is two vertex-disjoint paths $s_0 \to t_0$ and $s_1 \to t_1$ covering every cell. Two-pair instances have structure that a single pair lacks: the paths can block each other, and blocking can be forced from far away by the geometry of a corner. Our main result is that all such obstructions are local or global in a controlled way.

Theorem 1 (Main theorem)**.** *Let I be an instance on an $R \times C$ rectangle with $R, C \geq 11$. Then I is solvable if and only if* Passes (I)*, i.e. no entry of the catalogue of Section 5 fires. If* $\min(R, C) \leq 10$*, solvability is decided by a dynamic program in time linear in* $\max(R, C)$*. In both cases a solution, when one exists, is constructed in polynomial time.*

The catalogue test runs in time $O(R+C)$, so solvability is decided in linear time for both sides at least 11, and in time linear in the long side otherwise.

Contributions.

- A finite catalogue of forbidden patterns that characterizes solvability for both sides ≥ 11 (Section 5).
- A constructive algorithm: line deletions with a *band extension* lemma that lifts solutions back, straight cuts (*moves*) with gluing, and the plug DP for thin rectangles (Sections 6 and A).
- A Lean 4 formalization of both directions and of the thin case (Section 9), with the compromises made explicit.
- Implementations in Python, JavaScript and C, tested against each other, the Lean definitions and exact solvers (Section 8).

2 Definitions

Cells are (r, c) with $0 \leq r < R$, $0 \leq c < C$; two cells are adjacent if they differ by one in exactly one coordinate. A *line* is a full row or column; it is *free* if it contains no endpoint. A *gap* is a maximal run of free lines on one axis; an *edge gap* touches the border, an *internal gap* lies between two endpoint lines. A cell is *black* if $r + c$ is even, and $\sigma(x) = +1$ for black cells, -1 for white ones.

The instance is *thin* if $\min(R, C) \leq 10$; the *domain* $\mathcal{D}$ is the set of instances with both sides ≥ 11.

One-path pieces. For a rectangle and two cells $s \neq t$, IPS [3] show that a Hamiltonian s–t path exists iff (a) if RC is odd then s and t are both black, and if RC is even they have different colours; and (b) (s, t) is not in one of three explicit families occurring when a side has length 1, 2 or 3. We call such a pair *acceptable*. A Hamiltonian path can be constructed in linear time.

Cuts and moves. A *cut* is a straight line between two adjacent rows (or columns). Where a path crosses it, the two cells on either side form a *crossing pair*, and each becomes a *virtual endpoint* of its *piece*. A *move* is a cut together with a choice of crossings; it is *valid* if every piece is OK: a one-path piece is acceptable, and a two-path piece is either thin and solvable, or in $\mathcal{D}$ and passing the catalogue.

3 Related work

Hamiltonian paths in grid graphs. Itai, Papadimitriou and Szwarcfiter [3] proved the Hamiltonian path problem NP-complete in general grid graphs and gave the characterization for rectangles that we use for one-path pieces. Umans and Lenhart [7] showed that Hamiltonian cycles in solid grid graphs (without holes) can be found in polynomial time. Our setting adds a second pair on a rectangle; the one-pair case is exactly IPS.

Numberlink. Adcock, Demaine, Demaine, O'Brien, Reidl, Sánchez Villaamil and Sullivan [1] proved that Zig-Zag Numberlink is NP-complete. For a single pair the problem is the Hamiltonian path problem, solved on rectangles by IPS. The complexity with a fixed number $k \geq 2$ of pairs was, to our knowledge, open; this paper resolves $k = 2$ on rectangles. Without the covering requirement, the problem is a vertex-disjoint paths problem, NP-complete for unbounded k even in planar graphs [4] and polynomial for every fixed k in general graphs by the Graph Minors theory of Robertson and Seymour [5]; the covering requirement changes the problem substantially, since it forces the paths to fill the grid.

Frontier dynamic programs. The plug DP for thin rectangles is a transfer-matrix (frontier-based) dynamic program over connectivity states of a boundary, a standard technique for path and cycle problems on grids of bounded width.

4 Overview of the approach

Thin rectangles are handled exactly by the plug DP. For wide rectangles, the guiding observation is that obstructions are created only near the four corners and along the border; the interior of a large rectangle is flexible. This suggests:

1. *Obstructions* (Theorem B): find a catalogue of patterns, each of which provably prevents a solution. Most are short arguments (parity, planarity of the two paths, forced degrees at corners); some corner configurations are certified by exhaustive local computations.

2. *Reductions* (Theorem A, induction step): if an axis has a long run of free lines, delete two of them. Passing is preserved because the catalogue only looks at corner windows of bounded size and at the cyclic order of endpoints on the border; and any solution of the smaller instance extends to the larger one by *band extension*.

3. *Base* (Theorem A, base case): a side of length ≥ 23 always admits a reduction, so the irreducible instances have sides in $[11, 22]$. For each passing irreducible instance there is a valid move: a straight cut whose pieces are smaller and OK. This finite statement is checked by computer.

Theorems A and B together give the characterization; the reductions and moves give the construction.

5 The catalogue of forbidden features

The catalogue is closed under the eight symmetries of the rectangle, swapping colours, and swapping s with t within a colour; we list entries up to these symmetries. Corner entries are evaluated in the eight *corner frames* (four corners, two orientations), with frame coordinates (i, j) counted from the corner. Below a is a colour and $\bar{a}$ the other one.

The R entry. Some corner patterns force a path through the corner cell. Rule R1: an endpoint at $(0, 1)$ with $(0, 0)$ and $(1, 0)$ empty forces its path through $(0, 0)$ to $(1, 0)$, so the endpoint effectively sits at $(1, 0)$ and the cells $(0, 0), (0, 1)$ are settled. Rules R2 (a same-colour diagonal at $(0, 0), (1, 1)$) and R3 (a different-colour pair at $(1, 2), (2, 1)$) settle 2×2 and 3×3 corner regions similarly. After applying the rules in a fixed order, R fires if the effective endpoints alternate along the border of the remaining region (T1 for that region).

Entry	Fires when
P	$\sum \sigma(\text{endpoint}) \neq 2\,(RC \bmod 2)$ (the parity of two covering paths)
T1	all endpoints on the border, colours alternating A, B, A, B around it
T2	the endpoints fill a 2×2 square, each colour on a diagonal
L1	an endpoint whose in-grid neighbours are all endpoints of the other colour
L6	an empty corner cell whose neighbours are endpoints of one colour, for both colours
L2–L5	small corner patterns, e.g. L2: $(0,1) = a$, $(1,0) = \bar{a}$, $(0,0)$ empty
E1	along an edge: $(0,k) = (1,k{+}1) = a$, $(1,k{+}2) = (0,k{+}3) = \bar{a}$
B	a listed corner triple (21 triples, or 5 when both sides are odd), with the fourth endpoint on the border outside the 6×6 corner window
C, D	the four endpoints form one of 36 listed corner configurations (RC even) or 3 (RC odd), within an 8×8 corner window
R	after forced corner routes (rules R1–R3), the effective endpoints alternate along the border of the remaining region

Table 1: The catalogue. The lists for B, C, D are in Appendix B.

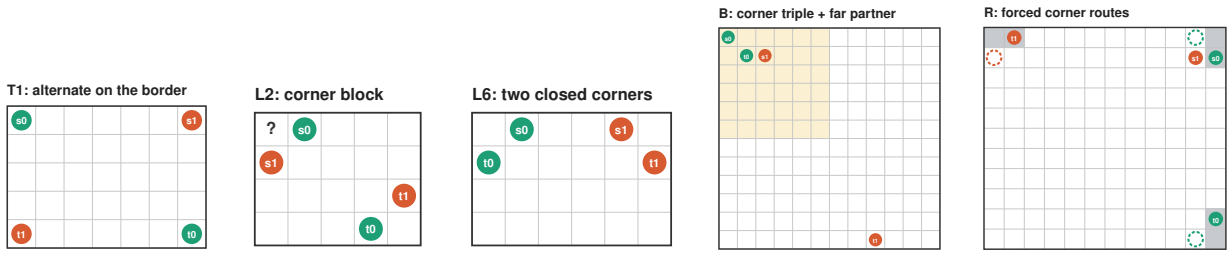


Figure 1: Entries T1, L2, L6, B and R. In R, grey cells are settled by forced corner routes and dashed circles mark the effective endpoints.

Motivation for the entries. P and T1 are the classical obstructions: parity (each path alternates colours) and planarity (two disjoint paths between alternating border points would cross). T2, L1–L6 and E1 express forced degrees near endpoints and corners. B, C and D are the residue: corner configurations that are unsolvable for reasons that are local but not short; each was found by exhaustive search on corner windows, and each is certified by a window computation (Section 7).

6 The algorithm

6.1 Reductions and band extension

On an axis of length $n \geq 13$, the following reductions delete two free lines $d, d+1$:

(S) *strip*: an edge gap of at least 4 lines (delete the outermost two);

(K) *compression*: an internal gap of at least 10 lines (delete two from its middle);

(I) *interior compression*: an internal gap of at least 3 lines containing $d, d{+}1$ with $8 \leq d \leq n{-}10$.

Lemma 2 (Band extension)**.** *If I' is solvable and row r of I' is free, then inserting two empty rows directly below r gives a solvable instance I. A solution of I is obtained from one of I' in time $O(RC)$.*

Construction. Stretch every path edge crossing below row r through the two new rows. The new rows split into blocks between consecutive crossings; each crossing absorbs the block on one of its sides by a U-shaped detour. The remaining block is absorbed by a *square flip*: a path edge (r, a)–$(r, a{+}1)$ inside it is replaced by the block's boundary cycle. Such an edge exists by counting: the m crossing cells of row r supply at most m horizontal edges, while each of the $m + 1$ runs of non-crossing cells needs one (its cells have degree two and no vertical edge downward). □

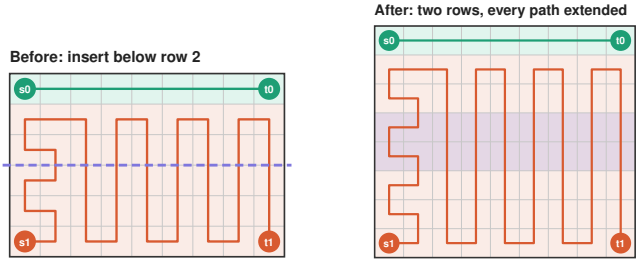


Figure 2: Band extension: two rows inserted below row 2.

6.2 Moves

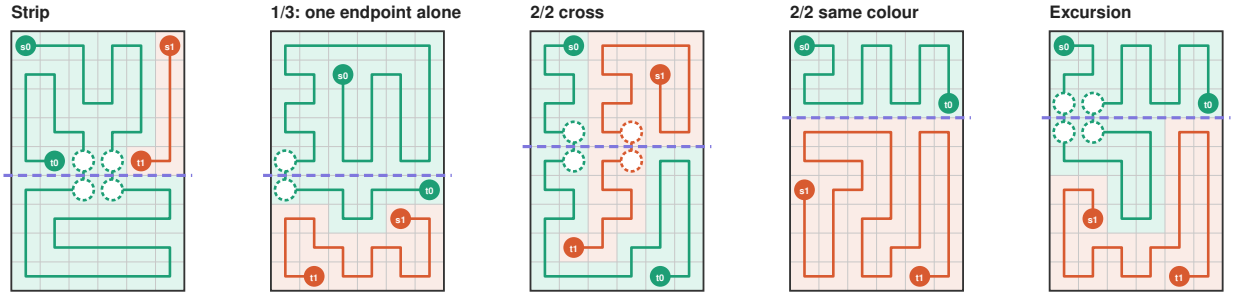


Figure 3: The five move types: strip (an empty block is spliced in), 1/3, 2/2 cross, 2/2 same, and excursion (colour A leaves its side and returns).

A move cuts the rectangle between rows $p-1$ and p (up to symmetry). The types, by how the endpoints fall (Figure 3): *strip* (all endpoints on one side; the other side is empty, has even area and at least two lines; its block is spliced into a path running along the cut); *1/3* (one endpoint alone; its piece is a one-path piece to its crossing); *2/2 cross* (one endpoint of each colour on each side; two crossings); *2/2 same* (each side holds one whole pair; two one-path pieces); *excursion* (each side holds one whole pair, and one colour crosses the cut twice). Pieces' solutions glue along the crossing edges.

6.3 Pseudocode

Algorithm 1 decides an instance, Algorithm 2 constructs a solution, and Algorithm 3 is the band extension used to undo a reduction.

Algorithm 1 Decision

Input: instance $I = (R, C, s_0, t_0, s_1, t_1)$ with four distinct endpoints
Output: whether I is solvable

if $\min(R, C) \le 10$ **then return** PlugDPFeasible(I) ▷ Appendix A
else return not Fires(I) ▷ Section 5

Algorithm 2 Construction

function Solve(I)
  if $\min(R, C) \le 10$ **then return** PlugDPSolve(I)
  if Fires(I) **then return** None
  if $(axis, d) \leftarrow$ FindReduction(I) succeeds **then**
    $I' \leftarrow I$ with lines $d, d+1$ on $axis$ deleted
    $S' \leftarrow$ Solve(I')
    return BandExtend(S', $axis$, d) ▷ Lemma 2
  if a cut separates the two pairs and both one-path pieces are acceptable **then**
    return the two Hamiltonian paths (IPS construction)
  $M \leftarrow$ FindMove(I) ▷ exists by Theorem A; here $R, C \le 22$
  solve each piece of M (by IPS, the plug DP, or Solve)
  return the pieces' solutions glued along M's crossings

function FindReduction(I)
  for $axis \in \{\text{rows}, \text{columns}\}$ with length $n \ge 13$ **do**
    if lines 0..3 are free **then return** $(axis, 0)$
    if lines $n{-}4..n{-}1$ are free **then return** $(axis, n-2)$
    if lines $\ell..\ell{+}9$ are free, with endpoints on both sides **then return** $(axis, \ell+4)$
    if $8 \le d \le n{-}10$, lines $d, d{+}1$ and $d{-}1$ or $d{+}2$ free, endpoints on both sides **then**
      return $(axis, d)$
  return None

function FindMove(I)
  for $m = 0, 1, \ldots, 10$ **do** ▷ width of the widest thin piece; 0: none
    for each move type, each of the 8 orientations, each endpoint relabelling **do**
      for each cut position p and crossing choice whose widest thin piece has width m **do**
        if every piece is OK (narrower piece checked first) **then return** the move

6.4 Complexity

Each reduction removes two lines, so at most $(R+C)/2$ reductions occur, each costing a catalogue check ($O(R+C)$) and a band extension ($O(RC)$). Moves occur only on rectangles with both sides at most 22, in constant time. The total is $O((R+C)\,RC)$ for $R, C \ge 11$, plus the plug DP, which is linear in the long side for a fixed short side.

Algorithm 3 Band extension (Lemma 2), rows

Input: a solution P of I'; a free row r
Output: a solution of I' with two empty rows $n_1 = r{+}1$, $n_2 = r{+}2$ inserted

1: shift every cell below row r down by two
2: $X \leftarrow$ columns c where P uses the edge (r, c)–$(r{+}3, c)$ ▷ crossings
3: **for** $c \in X$ **do** replace the edge by $(r, c), (n_1, c), (n_2, c), (r{+}3, c)$
4: let $B_0, \ldots, B_{|X|}$ be the column blocks of rows n_1, n_2 between crossings
5: $j \leftarrow$ an index with $B_j = \emptyset$, or else one whose block contains a path edge (r, a)–$(r, a{+}1)$
6: **for** the i-th crossing c **do** absorb B_{i-1} if $i \le j$, else B_i, by a U-detour replacing (n_1, c)–(n_2, c)
7: **if** $B_j \neq \emptyset$ **then** replace (r, a)–$(r, a{+}1)$ by the boundary cycle of B_j opened at (n_1, a)–$(n_1, a{+}1)$

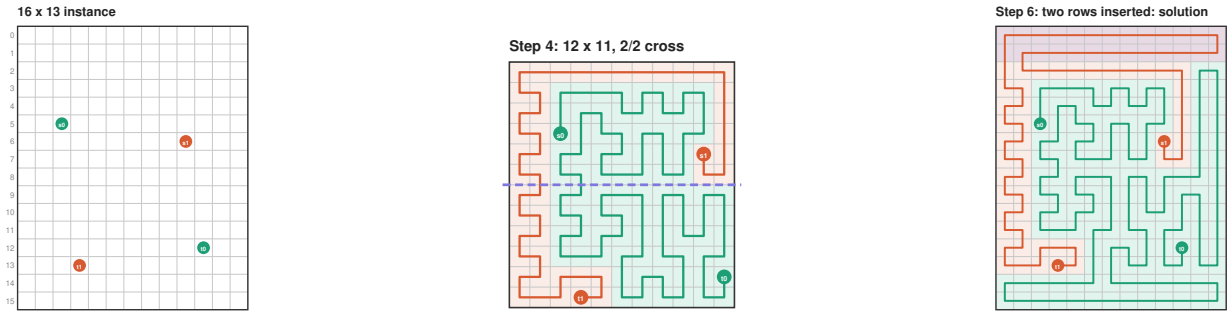


Figure 4: A 16×13 instance; the 12×11 core with its 2/2-cross move; the solution.

6.5 A worked example

For the 16×13 instance of Figure 4: rows 0–3 are free, so the strip reduction deletes rows $0, 1$; the 14×13 result has no reduction, and strip moves cut off a 2×13 block below and a 12×2 block on the right; the 12×11 core is split by a horizontal 2/2-cross move into two thin 6×11 pieces, solved by the plug DP. The blocks are spliced back and the two deleted rows reinserted by band extension.

7 Proof outline

Theorem 3 (Theorem B: soundness of the catalogue)**.** *If an entry fires on an instance in $\mathcal{D}$, the instance is unsolvable.*

P, T1, T2, L1–L6, E1 and R have direct proofs (parity; two disjoint paths between alternating border points of a disc cross; forced degrees). B, C and D are certified by *window certificates*: a dynamic program over the corner window enumerates all ways a solution can look inside it, and an outside test (parity, a non-crossing perfect matching of the boundary crossings, path structure) shows that none can be completed outside, for any rectangle in $\mathcal{D}$.

Theorem 4 (Theorem A: completeness of the catalogue)**.** *Every instance in $\mathcal{D}$ that passes the catalogue is solvable.*

By induction on the area. If a reduction applies, the smaller instance passes (each entry's view is unchanged) and is solvable by induction; band extension lifts its solution. Otherwise both sides are at most 22 (a side ≥ 23 always has a reduction); for these finitely many instances a valid move

exists (computer check), its thin pieces are solvable, its other two-path pieces are smaller passing instances in $\mathcal{D}$, solvable by induction, and the pieces glue.

8 Implementations

We provide libraries in Python (the reference), JavaScript and C, each with a command-line program. All three implement Algorithms 1–3 and the plug DP with the same iteration orders, and produce byte-identical output, including the paths. Testing:

- the catalogue of each implementation agrees with the Lean definition on 60,000 random instances (sides 2–24, endpoints clustered near corners);
- the three implementations agree on 450 random instances with sides 3–45, and every reported solution is verified;
- the C implementation solves 20,000 random instances with sides 11–60 in about 8 seconds, with DECIDE and SOLVE agreeing on every instance;
- on 300 random instances with sides 11–12, DECIDE agrees with an independent exact solver.

9 Verification in Lean

The development is about 27,000 lines of Lean 4 [2] with Mathlib [6]. The trusted statement is a short file defining instances, paths and solutions. The main results:

- `passes_iff_solvable`: for $R, C \geq 11$ and a well-formed instance, PASSES$(I) \leftrightarrow$ Solvable(I);
- `thinBF_iff`: the (proof-oriented) plug DP accepts iff the instance is solvable, for every rectangle, using only the standard axioms;
- `solvableB_iff`: a decision procedure for every rectangle is correct.

Here PASSES is a computable Boolean function; the theorem is about it as defined, so no informal transcription of the catalogue needs to be trusted. The IPS characterization is formalized as well and used for one-path pieces.

Compromises.

1. *Computation by `native_decide`.* The window certificates (10 theorems) and the base case (144 theorems, one per box of sides 11–22) are evaluated by the Lean compiler rather than the kernel. This adds 154 axioms that trust the compiler and the generated native code. Kernel evaluation is not practical: on a 6×6 window the kernel exceeded 16 GB after 20 minutes on a computation the compiled code does in 3 seconds, and an 8×8 window reaches 2.3×10^5 states.
2. *Symmetry handling.* The base case checks one representative per orbit of the 64-element symmetry group (geometric symmetries, colour swap, reversing either path). Instead of proving that the catalogue is invariant under reflections, the computation checks that the representative or one of its reflections passes; invariance under the other symmetries is proved. The search region is about 2.8×10^{10} instances; the check takes about 16 CPU-hours (4 hours on 4 threads).

3. *Checked search instead of verified search.* The move search is a Boolean function proved sound (a found move is valid); thin pieces are solved by an unverified solver and the solutions are checked by a verified checker.

4. *Precompiled modules.* Without `precompileModules`, `native_decide` interprets the project's functions and is about 400× slower.

5. *Running time.* The polynomial bound is not formalized; Lean has no standard cost model short of Turing machines. The theorems certify what is computed; the running-time analysis is on paper.

6. *Thin case.* The verified plug DP uses a proof-friendly state encoding (sorted lists of pieces coded by their ends) rather than the bracket encoding of the implementations.

10 Discussion

The characterization gives a linear-time decision procedure for two pairs on large rectangles, in contrast with NP-completeness for unbounded k. Natural questions are three or more pairs on rectangles, where corner interactions multiply, and two pairs on other regions such as solid grid graphs. The catalogue's corner configurations (B, C, D) were found by computer; a conceptual description of them would shorten the proof and might allow kernel-checked certificates.

References

[1] Aaron B. Adcock, Erik D. Demaine, Martin L. Demaine, Michael P. O'Brien, Felix Reidl, Fernando Sánchez Villaamil, and Blair D. Sullivan. Zig-zag Numberlink is NP-complete. *Journal of Information Processing*, 23(3):239–245, 2015. arXiv:1410.5845.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, 2021.

[3] Alon Itai, Christos H. Papadimitriou, and Jayme Luiz Szwarcfiter. Hamilton paths in grid graphs. *SIAM Journal on Computing*, 11(4):676–686, 1982.

[4] James F. Lynch. The equivalence of theorem proving and the interconnection problem. *ACM SIGDA Newsletter*, 5(3):31–36, 1975.

[5] Neil Robertson and Paul D. Seymour. Graph minors. XIII. the disjoint paths problem. *Journal of Combinatorial Theory, Series B*, 63(1):65–110, 1995.

[6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

[7] Christopher Umans and William Lenhart. Hamiltonian cycles in solid grid graphs. In *Proceedings of the 38th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 496–505, 1997.

A The plug DP

Transpose so that the grid has $W \le L$ rows. Cells are processed column by column, top to bottom. The *frontier* after some cells consists of $W+1$ edges between processed and unprocessed cells: one per row, plus the edge below the current cell (Figure 5). A state assigns each frontier edge a 3-bit code: unused; an *anchor* of colour A or B (a piece ending at an endpoint); or an *open* or *close* bracket of colour A or B (a piece with both ends on the frontier; matched like parentheses, since pieces cannot cross).

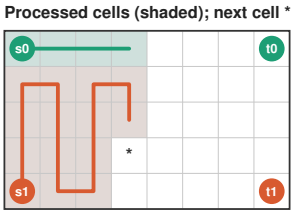


Figure 5: Processed cells (shaded) and the next cell ($*$).

Algorithm 4 Plug DP (decision)

function PLUGDPFEASIBLE(I) ▷ $W \le L$ after transposition
  $S \leftarrow$ {all-empty}
  for column $c = 0, \dots, L-1$ **do**
    if $c > 0$ **then** shift every state by one slot
    for row $r = 0, \dots, W-1$ **do**
      $S \leftarrow \bigcup_{s \in S} \text{STEP}(s, r, c)$
  return all-empty $\in S$

function STEP(s, r, c) ▷ slot r: plug from above; r+1: from the left
  $b \leftarrow 1$ if (r, c) is an endpoint of colour k, else 2; $h \leftarrow$ number of incoming plugs
  if $h > b$ or $b - h$ exceeds the free directions (right, down) **then return** $\emptyset$
  if $h = 0$ **then**
    at an endpoint: a new anchor of colour k, going right or down
    otherwise: a new bracket pair of either colour, going right and down
  else if $h = 1$ **then**
    continue the plug right or down, or end it if (r, c) is an endpoint of its colour
    (ending an anchor completes its path; ending a bracket makes its partner an anchor)
  else ▷ join two plugs of the same colour
    two anchors: path complete; an anchor and a bracket: the partner becomes an anchor
    close then open: the pieces merge; open then close: a cycle, rejected
    two opens or two closes: the outer partner's bracket type changes
  return the resulting states (outgoing plugs in slots r = right, r+1 = down)

Construction. The forward pass stores one checkpoint (the state set) per column. A backward pass re-runs each column from its checkpoint, recording predecessors and keeping only states whose

processed slots agree with the known end-of-column state, then traces back the chosen edges. The number of states is bounded by the number of valid frontiers (about 10^5 for $W = 10$), so the running time is $O(L \cdot W \cdot \#\text{states})$.

Correctness. By induction over the cells, the reachable states are exactly the signatures (used frontier edges, their pairing, and the colours of pieces ending at endpoints) of partial solutions on the processed cells; hence the empty final state is reachable iff a solution exists. Lean verifies this for an equivalent DP with a different encoding.

B The listed corner configurations

Cells are written ij for frame coordinates (i, j).
B (RC even; triples $(p\,q\,r)$: colour a at p, q, $\bar{a}$ at r): (00 11 12), (01 02 11), (01 02 12), (01 03 11), (01 04 11), (01 04 12), (01 05 11), (01 12 11), (01 12 13), (01 12 22), (01 12 31), (01 13 03), (01 21 13), (01 21 22), (01 21 31), (02 21 10), (03 11 10), (04 11 10), (04 21 10), (04 21 12), (05 11 10).
B (R, C odd): (00 11 12), (00 11 22), (01 02 11), (01 04 11), (04 11 10).
C (RC even; {one pair | the other pair}): {01 03 | 11 13}, {01 06 | 07 11}, {01 06 | 11 70}, {01 06 | 12 60}, {01 07 | 11 60}, {01 13 | 11 12}, {01 13 | 12 22}, {01 13 | 12 31}, {01 22 | 11 12}, {01 22 | 12 31}, {01 22 | 13 30}, {01 70 | 11 60}, {02 11 | 21 30}, {02 20 | 12 21}, {03 21 | 31 40}, {06 21 | 12 60}, {12 22 | 13 21}, {12 31 | 13 21}, {01 12 | 04 22}, {01 12 | 04 31}, {01 12 | 13 20}, {01 12 | 13 40}, {01 12 | 20 22}, {01 12 | 22 40}, {01 12 | 31 40}, {01 13 | 03 04}, {01 13 | 03 20}, {01 13 | 03 40}, {01 21 | 02 22}, {01 21 | 02 31}, {01 21 | 04 13}, {01 21 | 04 22}, {01 21 | 04 31}, {01 21 | 13 40}, {01 21 | 22 40}, {01 21 | 31 40}.
D (RC odd): {00 01 | 11 20}, {00 14 | 02 13}, {00 23 | 02 13}.